



Trabalho de Projeto

2ª Fase

1 Descrição Geral

A componente teórico-prática da disciplina de sistemas distribuídos consiste no desenvolvimento de quatro projetos, utilizando a linguagem de programação C [1], sendo que a realização de cada um deles é necessária para a realização do projeto seguinte. Por essa razão, **é muito importante que consigam ir cumprindo os objetivos de cada projeto, de forma a não hipotecar os projetos seguintes.**

O projeto consiste em implementar um sistema de gestão de inventário para um conjunto de um stand automóvel, utilizando uma lista encadeada para armazenar a informação dos carros.

No projeto 1 foram definidas estruturas de dados e implementadas várias funções para lidar com a manipulação dos dados que vão ser armazenados na lista [2]. Também foi construído um módulo para serialização de dados, que teve como objetivo familiarizar os alunos com a necessidade de serializar dados para a comunicação em aplicações distribuídas.

O projeto 2 tem como objetivo implementar um sistema cliente-servidor simples, sendo o servidor responsável por manter uma lista (usando os módulos construídos no projeto 1) e sendo o cliente responsável por comunicar com o servidor para realizar operações na lista. Neste projeto, em vez de se utilizar o módulo de serialização desenvolvido no projeto 1 (que teria de ser consideravelmente estendido com outras funções), serão utilizados os **Protocol Buffers** da Google [3], que permitem automatizar a criação das funções para **serializar** (codificar) e **de-serializar** (descodificar) os dados a serem enviados/recebidos, a partir de um ficheiro com a descrição destes dados.

Será, portanto, necessário programar duas aplicações, um servidor e um cliente, sendo o servidor responsável por manter uma lista em memória, receber pedidos de um cliente para executar na lista e responder ao cliente, e sendo o cliente responsável por apresentar uma interface ao utilizador, enviando comandos dados por este para o servidor, e apresentando os resultados recebidos. Em mais detalhe, isto implica que:

1. através do cliente, o utilizador irá invocar operações, que serão transformadas em mensagens e enviadas pela rede até ao servidor;
2. este, por sua vez interpretará essas mensagens, e;
 - a. realizará as operações correspondentes na lista local;
 - b. enviará depois ao cliente a resposta transformada em mensagem;
3. por sua vez, o cliente interpretará a mensagem de resposta e;
4. apresentará o resultado ao utilizador, ficando depois pronto para a próxima operação.

O objetivo final é fornecer às aplicações que usariam esta lista um modelo de comunicação tipo RPC (*Remote Procedure Call*)¹, onde vários clientes acedem à mesma lista partilhada. Neste projeto apenas será atendido um cliente de cada vez, sendo que nos próximos projetos o sistema passará a suportar vários clientes em simultâneo. Para a concretização deste projeto 2 será necessário combinar o código desenvolvido no projeto 1 com os *Protocol Buffers* e com as funções de sistema para comunicação usando *sockets* TCP.

Espera-se uma grande fiabilidade por parte do servidor, portanto **todas as condições de erro devem ser tratadas**. É igualmente necessário garantir que a gestão de memória no cliente e especialmente no servidor é feita de forma correta (evitando *memory leaks* ou acesso a zonas de memória inválidas). É importante notar que em sistemas cliente-servidor os servidores ficam em funcionamento permanente durante muito tempo (dias, meses ou até anos) e não é suposto que parem (*crash*) por causa de erros. No caso da *Amazon*, se o serviço que mantém as cestas de compras dos clientes não funciona, a empresa não vende, e perde milhões por hora.

2 Descrição específica

O objetivo específico do projeto 2 é desenvolver uma aplicação do tipo cliente-servidor, utilizando o paradigma das chamadas a procedimentos remotos (RPC – *Remote Procedure Calls*) para concretizar a interação entre o cliente e o servidor. O programa cliente deverá implementar uma interface com o utilizador, permitindo que este solicite a execução de operações numa lista. Esta lista será implementada no programa servidor (utilizando o código desenvolvido no projeto 1), que deverá oferecer um conjunto de procedimentos acessíveis remotamente, para executar determinadas operações na lista. Assim, o cliente terá de ser programado para enviar para o servidor as operações solicitadas pelo utilizador, receber e apresentar as respostas. Por seu lado, o servidor terá de ser programado para esperar um pedido de ligação de um cliente, ficando depois pronto para receber, executar e

¹ Ver aula teórica sobre RPC e capítulo 4 do livro texto da cadeiaira.

responder às operações enviadas por esse cliente, até que este se desligue (podendo depois atender um novo cliente). Em termos práticos, o projeto 2 consiste na concretização de:

1. Dois programas, escritos na linguagem C, a serem executados da seguinte forma:
 - a. **list_server** <port>
 <port> é o número do porto TCP ao qual o servidor se deve associar (fazer *bind*).
 - b. **list_client** <server>:<port>
 <server> é o endereço IP ou nome do servidor da lista.
 <port> é o número do porto TCP onde o servidor está à espera de ligações.
2. Módulos de adaptação do lado do cliente (*stub*) e do lado do servidor (*skeleton*) que permitam preparar as mensagens que são serializadas para envio, e processar o conteúdo das mensagens recebidas após terem sido de-serializadas. Para a serialização e de-serialização, será fornecido um ficheiro *.proto* com a especificação do formato genérico das mensagens que serão usadas para a comunicação entre cliente e servidor, a partir do qual serão gerados, usando os *Protocol Buffers*, os ficheiros *.c/.h* que definem, em C, o formato das mensagens e as funções necessárias para serializar (codificar) e de-serializar (descodificar) as mesmas.

O programa cliente (**list_client**) permitirá o envio de comandos via rede para invocar operações (na lista) implementadas pelo servidor. Por outro lado, o programa servidor (**list_server**) receberá os comandos vindos dos clientes, executando-os sobre a lista e devolvendo a resposta ao cliente. Estes comandos são constituídos por *opcodes* representativos de *operações* (OP_**) e *tipo de conteúdo* (CT_**) a operar na lista.

A Tabela 1 apresenta os comandos e respetivas sintaxes, que os clientes podem requerer e a que o servidor pode responder.

COMANDO UTILIZADOR	MENSAGEM DE PEDIDO	MENSAGEM DE RESPOSTA
add <data>	OP_ADD CT_DATA <data>	OP_ADD+1 CT_NONE OP_ERROR CT_NONE
get_by_marca <marca>	OP_GET CT_MARCA <marca>	OP_GET+1 CT_VALUE <value> OP_ERROR CT_NONE
get_by_year <year>	OP_GET CT_YEAR <year>	OP_GET+1 CT_YEAR <car> OP_ERROR CT_NONE
get_model_list	OP_GETMODELS CT_NONE	OP_GETMODELS+1 CT_MODEL <model> OP_ERROR CT_NONE
get_list_ordered_by_year	OP_GETLISTBYTEAR CT_NONE	OP_GETLISTBYTEAR +1 CT_LIST <cars> OP_ERROR CT_NONE
remove <model>	OP_DEL CT_MODEL <model>	OP_DEL+1 CT_NONE OP_ERROR CT_NONE
size	OP_SIZE CT_NONE	OP_SIZE+1 CT_RESULT <size> OP_ERROR CT_NONE
quit	--	--

Tabela 1: Definição dos comandos e respetivas mensagens de pedido e de resposta.

Caso a operação sobre a lista seja bem-sucedida, o *opcode* de resposta é igual ao *opcode* do pedido incrementado de uma unidade. Por outro lado, caso a operação no servidor não seja bem-sucedida, o *opcode* da resposta será OP_ERROR.

É assim necessário, nos programas cliente e servidor, construir mensagens que respeitem este formato. Para facilitar a definição das estruturas de dados que vão constituir uma mensagem, e para automatizar o processo de serialização dos dados (mensagem a enviar) num buffer, e de-serialização do buffer nos correspondentes dados (mensagem recebida), serão usados *protocol buffers*.

Em concreto, para se usar os *protocol buffers* é necessário, em primeiro lugar, definir o formato das mensagens a enviar/receber. Esta definição é feita num ficheiro *.proto*, usando a sintaxe “proto3”. Neste projeto, a definição será feita no ficheiro *sdmessage.proto*, que será fornecido a todos os grupos. Desta forma, todos os grupos terão de seguir a mesma definição do formato das mensagens, pelo que **será de esperar que qualquer programa cliente**

consiga interagir com qualquer programa servidor. Estando assim assegurada a interoperabilidade entre qualquer cliente e qualquer servidor que implemente corretamente a interface definida. Serão também disponibilizados binários de um cliente e de um servidor, que os grupos poderão usar para testar, respetivamente, o seu próprio servidor ou cliente. Estes binários funcionarão no ambiente Linux dos laboratórios do DI.

De seguida é apresentado o ficheiro `sdmessage.proto`.

```
/* Ficheiro sdmessage.proto */

syntax = "proto3";

enum Marca {
    MARCA_TOYOTA = 0;
    MARCA_BMW = 1;
    MARCA_RENAULT = 2;
    MARCA_AUDI = 3;
    MARCA_MERCEDES = 4;
}

enum Combustivel {
    COMBUSTIVEL_GASOLINA = 0;
    COMBUSTIVEL_GASOLEO = 1;
    COMBUSTIVEL_ELETRICO = 2;
    COMBUSTIVEL_HIBRIDO = 3;
}

message Data {
    int32 ano = 1;
    float preco = 2;
    Marca marca = 3;
    string modelo = 4;
    Combustivel combustivel = 5;
}

message message_t {
    enum Opcode {
        OP_BAD = 0;
        OP_ADD = 10;
        OP_GET = 20;
        OP_DEL = 30;
        OP_SIZE = 40;
        OP_GETMODELS = 50;
        OP_GETLISTBYTEAR = 60;
        OP_ERROR = 99;
    }
    Opcode opcode = 1;

    enum C_type {
        CT_BAD = 0;
        CT_DATA = 10;
        CT_MARCA = 20;
        CT_YEAR = 30;
        CT_MODEL = 40;
        CT_RESULT = 50;
        CT_LIST = 60;
        CT_NONE = 70;
    }
    C_type c_type = 2;

    Data data = 3;
    sint32 result = 6;
    repeated string models = 7;
    repeated Data cars = 8;
}
```

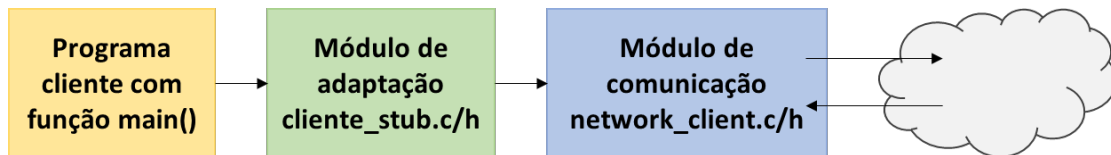
Para além de ser fornecido o ficheiro `sdmessage.proto` e os binários do cliente e do servidor, para guiar a concretização dos programas cliente e servidor são ainda fornecidos ficheiros `.h` com os cabeçalhos de algumas funções que devem ser concretizadas. Os ficheiros fornecidos, incluindo o ficheiro `sdmessage.proto` e os ficheiros `sdmessage.pb-c.c/ sdmessage.pb-c.h` que dele resultam, **não podem ser alterados**. As concretizações das funções definidas nos ficheiros `X.h` devem ser feitas nos respetivos ficheiros `X.c`, utilizando os algoritmos e métodos que o grupo achar convenientes. É possível criar ficheiros `X-private.h` para acrescentar outras definições que o grupo entenda necessárias. Os ficheiros `.h` apresentados neste documento serão disponibilizados na página da disciplina.

3 Cliente

O cliente usa alguns dos módulos já desenvolvidos no projeto 1 (`data.c/h`), juntamente com três módulos adicionais:

- Programa cliente com a função `main()` (`list_client.c`)
- Módulo de adaptação das chamadas do cliente, RPC *stub* (`client_stub.c/h`)
- Módulo de comunicação (`network_client.c/h`)

A figura abaixo ilustra a interação entre os três módulos, onde o **módulo de adaptação** do cliente, isto é, o **RPC *stub*** (`client_stub.c/h`) permite a **interação entre o programa cliente e o módulo de comunicação**, tendo como função fazer com que **todos os detalhes relativos à comunicação sejam transparentes para o cliente**, ou seja, escondidos deste.



3.1 Programa cliente: `list_client.c`

O programa cliente consiste num programa interativo simples, que quando executado aceita um comando (uma linha) do utilizador no *stdin*, invoca a chamada remota através da respetiva função do *stub* (Secção 3.2), imprime a resposta recebida no ecrã e volta a aceitar um novo comando. **Uma boa forma de ler e tratar os comandos inseridos é usando as funções *fgets* e *strtok***. Cada comando vai ser inserido pelo utilizador numa única linha, devendo ser aceites os seguintes comandos (ver 1ª coluna da Tabela 1):

```
add <data>
remove <model>
get_by_year <year>
get_by_marca <marca>
get_list_by_year <year>
size
get_model_list
quit
```

3.2 RPC *stub*: `client_stub.c`

O *stub* implementa e disponibiliza uma função de adaptação para cada operação remota que o cliente possa efetuar no servidor, bem como funções para estabelecer e terminar a ligação a um servidor. Cada função de adaptação vai preencher uma estrutura `MessageT` com o tipo da operação (`opcode`), com o tipo de conteúdo (`c_type`) necessário para realizar a operação, e com esse conteúdo. Esta estrutura é então enviada para o módulo de comunicação, que a irá processar e devolver uma resposta (veja a próxima secção).

De notar que a estrutura `MessageT` é definida automaticamente pelo *Protocol Buffers* a partir do ficheiro `sdmessage.proto` e a definição estará no ficheiro `sdmessage.pb-c.h`,

A interface a ser oferecida ao cliente é apresentada de seguida.

```
#ifndef _CLIENT_STUB_H
#define _CLIENT_STUB_H

#include "data.h"
#include "list.h"

/* Remote list, deve conter as informações necessárias para comunicar
 * com o servidor. A definir pelo grupo em client_stub-private.h
 */

struct rlist_t; /* Definida em list-private.h */

/* Função para estabelecer uma associação entre o cliente e o servidor,
 * em que address_port é uma string no formato <hostname>:<port>.
```

```

* Retorna a estrutura rlist preenchida, ou NULL em caso de erro.
*/
struct rlist_t *rlist_connect(char *address_port);

/* Termina a associação entre o cliente e o servidor, fechando a
* ligação com o servidor e libertando toda a memória local.
* Retorna 0 se tudo correr bem, ou -1 em caso de erro.
*/
int rlist_disconnect(struct rlist_t *rlist);

/* Adiciona um novo carro à lista remota.
* O carro é inserido na última posição da lista.
* Retorna 0 (OK) ou -1 em caso de erro.
*/
int rlist_add(struct list_t *list, struct data_t *car);

/* Remove da lista remota o primeiro carro que corresponda ao modelo indicado.
* Retorna 0 se encontrou e removeu, 1 se não encontrou, ou -1 em caso de erro.
*/
int rlist_remove_by_model(struct list_t *list, const char *modelo);

/* Obtém o primeiro carro que corresponda à marca indicada.
* Retorna ponteiro para os dados ou NULL se não encontrar ou em caso de erro.
*/
struct data_t *rlist_get_by_marca(struct list_t *list, enum marca_t marca);

/* Obtém um array de ponteiros para carros de um determinado ano.
* O último elemento do array é NULL.
* Retorna o array ou NULL em caso de erro.
*/
struct data_t **rlist_get_by_year(struct list_t *list, int ano);

/* Ordena a lista remota de carros por ano de fabrico (crescente).
* Retorna 0 (OK) ou -1 em caso de erro.
*/
int rlist_order_by_year(struct list_t *list);

/* Retorna o número de carros na lista remota ou -1 em caso de erro.
*/
int rlist_size(struct list_t *list);

/* Constrói um array de strings com os modelos dos carros na lista remota.
* O último elemento do array é NULL.
* Retorna o array ou NULL em caso de erro.
*/
char **rlist_get_model_list(struct list_t *list);

/* Liberta a memória ocupada pelo array de modelos.
* Retorna 0 (OK) ou -1 em caso de erro.
*/
int rlist_free_model_list(char **models);

#endif

```

É importante lembrar que quando uma operação é executada pelo servidor com sucesso, o *opcode* para a mensagem de resposta deve ter o valor (*opcode* do pedido + 1). E sempre que ocorrer um erro do lado do servidor, este deverá enviar uma resposta com *opcode* `OP_ERROR` e conteúdo `CT_NONE`.

3.3 Módulo de comunicação do cliente: `network_client.c`

O módulo `network-client.c` vai concretizar as funções para estabelecer e terminar uma ligação ao servidor e, na função `network_send_receive()`, vai enviar a mensagem de pedido e receber a mensagem de resposta.

De forma mais detalhada, o que **tem de ser feito** nesta função é o seguinte:

1. Determinar a dimensão do buffer onde serão colocados os dados a enviar, com base na mensagem contida na estrutura `MessageT` que é recebida como argumento;
2. Serializar a mensagem contida na estrutura, colocando-a num buffer com dimensão apropriada;
3. Enviar um **short (2 bytes)** indicando a dimensão do buffer que será enviado;
4. Enviar o buffer;
5. Receber um **short** indicando a dimensão do buffer onde será recebida a resposta;
6. Receber a resposta colocando-a num buffer de dimensão apropriada;
7. De-serializar a resposta para uma estrutura `MessageT` que será retornada.

O módulo de comunicação `network_client.c` tem a seguinte interface:

```
#ifndef _NETWORK_CLIENT_H
#define _NETWORK_CLIENT_H

#include "client_stub.h"
#include "sdmessage.pb-c.h"

/* Esta função deve:
 * - Obter o endereço do servidor (struct sockaddr_in) com base na
 *   informação guardada na estrutura rlist;
 * - Estabelecer a ligação com o servidor;
 * - Guardar toda a informação necessária (e.g., descritor do socket)
 *   na estrutura rlist;
 * - Retornar 0 (OK) ou -1 (erro).
 */
int network_connect(struct rlist_t *rlist);

/* Esta função deve:
 * - Obter o descritor da ligação (socket) da estrutura rlist_t;
 * - Serializar a mensagem contida em msg;
 * - Enviar a mensagem serializada para o servidor;
 * - Esperar a resposta do servidor;
 * - De-serializar a mensagem de resposta;
 * - Tratar de forma apropriada erros de comunicação;
 * - Retornar a mensagem de-serializada ou NULL em caso de erro.
 */
MessageT *network_send_receive(struct rlist_t *rlist, MessageT *msg);

/* Fecha a ligação estabelecida por network_connect().
 * Retorna 0 (OK) ou -1 (erro).
 */
int network_close(struct rlist_t *rlist);

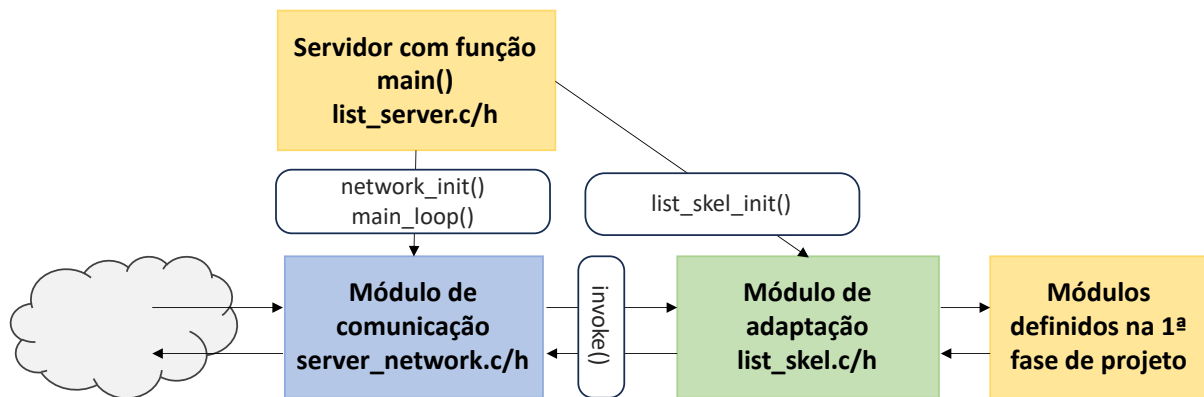
#endif
```

4 Servidor

O **servidor** a implementar usa os módulos `data.c/h` e `list.c/h` e é composto por três partes adicionais:

- Aplicação servidor (`list_server.c`) com a função `main()`
- Módulo de comunicação (`network_server.c/h`)
- Módulo de adaptação do servidor, i.e., RPC *skeleton* (`list_skel.c/h`)

A figura abaixo ilustra a interação entre os três módulos, onde o **módulo de adaptação** do servidor permite a **interação entre o módulo de comunicação e a lista**, permitindo também a **inicialização da lista** (feita na função `main()`).



4.1 Aplicação servidor: `list_server.c`

O servidor concretiza uma lista que pode ser acedida no porto definido na linha de comando. O servidor deverá suportar apenas um cliente de cada vez. Como o servidor apenas necessita de interagir com um cliente de cada vez, não será necessário, nesta fase, recorrer à criação de processos ligeiros (*threads*) para atendimento simultâneo de vários clientes.

A aplicação deve fazer essencialmente três coisas: inicializar o servidor, inicializar a lista, e colocar o servidor em *loop* a receber pedidos de ligação e a receber e responder a pedidos de operações na lista.

4.2 Módulo de Comunicação do servidor: `network_server.c`

O módulo `network_server.c` vai permitir inicializar o servidor (que deverá usar *sockets* TCP) e fechar o *socket* do servidor, respetivamente através das funções `network_server_init()` e `network_server_close()`. A função `network_main_loop()` é responsável pela interação com a rede (receber pedidos de ligação, receber pedidos de operações na lista e enviar respostas) e pela interação com o *skeleton* (`list_skel.c`).

Assim sendo, a função recebe sequências de bytes que constituem as mensagens com os pedidos dos clientes, realiza a de-serialização destes bytes, construindo uma mensagem (estrutura `MessageT`), a qual é entregue ao *skeleton* para processamento. No retorno, a mensagem enviada ao *skeleton* vem preenchida com o resultado da operação (ou seja, a estrutura `MessageT` é reaproveitada e alterada pelo *skeleton*), tendo de ser serializada numa sequência de bytes que são enviados ao cliente através da ligação TCP.

Tal como do lado do cliente, também o servidor **tem de receber/enviar um short (2 bytes)** antes receber/enviar as respetivas mensagens, para se saber a dimensão das mensagens (e dos *buffers* onde serão recebidas). Não deve ser esquecida a necessidade de libertar toda a memória reservada que já não é necessária, para evitar *memory leaks*!

O módulo de comunicação `network_server.c` tem a seguinte interface:

```
#ifndef _NETWORK_SERVER_H
#define _NETWORK_SERVER_H

#include "list.h"
#include "sdmessage.pb-c.h"

/* Função para preparar um socket de receção de pedidos de ligação
 * num determinado porto.
 * Retorna o descritor do socket ou -1 em caso de erro.
 */
int network_server_init(short port);

/* A função network_main_loop() deve:
 * - Aceitar uma conexão de um cliente;
 * - Receber uma mensagem usando a função network_receive;
 * - Entregar a mensagem de-serializada ao skeleton para ser processada
```

```

    na lista;
* - Esperar a resposta do skeleton;
* - Enviar a resposta ao cliente usando a função network_send.
* A função não deve retornar, a menos que ocorra algum erro. Nesse
* caso retorna -1.
*/
int network_main_loop(int listening_socket, struct list_t *list);

/* A função network_receive() deve:
* - Ler os bytes da rede, a partir do client_socket indicado;
* - De-serializar estes bytes e construir a mensagem com o pedido,
*   reservando a memória necessária para a estrutura MessageT.
* Retorna a mensagem com o pedido ou NULL em caso de erro.
*/
MessageT *network_receive(int client_socket);

/* A função network_send() deve:
* - Serializar a mensagem de resposta contida em msg;
* - Enviar a mensagem serializada, através do client_socket.
* Retorna 0 (OK) ou -1 em caso de erro.
*/
int network_send(int client_socket, MessageT *msg);

/* Liberta os recursos alocados por network_server_init(), nomeadamente
* fechando o socket passado como argumento.
* Retorna 0 (OK) ou -1 em caso de erro.
*/
int network_server_close(int socket);

/* Sinaliza término e fecha FDs internos (desbloqueia accept/read)*/
void network_server_request_shutdown(void);

#endif

```

4.3 RPC skeleton: list_skel.c

O *skeleton*, a concretizar em `list_skel.c`, serve para transformar uma mensagem do cliente (a qual foi de-serializada pela camada de rede) numa chamada (ou possivelmente em várias, caso o pedido não possa ser realizado com apenas uma chamada) da respetiva função do módulo `list.c`. Depois de realizar o pedido e de, possivelmente, obter dados de resposta, o *skeleton* deve reutilizar a estrutura `MessageT` que recebeu, para preparar a resposta a enviar de volta à camada de rede.

As funções `list_skel_init()` e `list_skel_destroy()` servem fundamentalmente para criar e destruir a lista a ser mantida pelo servidor. A lista devolvida pela função `list_skel_init()` deve ser passada para a função `network_main_loop()`, que a utilizará quando invocar as operações através da função `invoke()`.

```

#ifndef _LIST_SKEL_H
#define _LIST_SKEL_H

#include "sdmessage.pb-c.h"
#include "list.h"

/* Inicia o skeleton da lista.
* O main() do servidor deve chamar esta função antes de poder usar a
* função invoke().
* Retorna a tabela criada ou NULL em caso de erro.
*/
struct list_t *list_skel_init();

```



```

/* Liberta toda a memória ocupada pela lista e todos os recursos
 * e outros recursos usados pelo skeleton.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int list_skel_destroy(struct list_t *list);

/* Executa na lista a operação indicada pelo opcode contido em msg
 * e utiliza a mesma estrutura MessageT para devolver o resultado.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int invoke(MessageT *msg, struct list_t *list);

#endif

```

A função *invoke()* recebe a mensagem de-serializada numa estrutura *MessageT*, interpreta o campo *opcode* da mensagem para seleccionar a operação a executar na lista, e realiza a operação (ou as várias operações necessárias) na lista (também recebida como argumento). Depois de executar a operação, coloca o resultado na mesma estrutura *MessageT*, alterando os campos *opcode* e *c_type*, bem como os que forem necessários para enviar a resposta ao cliente.

Caso a operação na lista tenha sido executada com sucesso, o *opcode* da estrutura *MessageT* será o valor do *opcode* do pedido incrementado de uma unidade e o campo *c_type* terá o valor adequado ao tipo de resultado retornado. Por outro lado, caso ocorra algum erro na operação executada na lista, o campo *opcode* receberá o valor de *OP_ERROR* e o campo *c_type* o valor de *CT_NONE* (ver Tabela 1).

5 Observações

Algumas observações e dicas úteis:

- Recomenda-se a criação de funções *read_all()* e *write_all()* que vão receber e enviar *buffers* de qualquer dimensão pela rede, assegurando que só retornam quando o buffer for totalmente recebido ou enviado. Isto é necessário (ou pelo menos importante) dado que as funções *read()/write()* em sockets nem sempre leem/escrevem tudo o que lhes é pedido. Um bom sítio para concretizar essas funções é num módulo adicional *message-private.c* a ser usado quer do lado do cliente, quer do lado do servidor, com o protótipo das funções definido em *message-private.h*.
- Usar a função *signal()* para ignorar sinais do tipo *SIGPIPE*, lançados quando uma das pontas comunicantes fecha o *socket* de maneira inesperada. Isto deve ser feito tanto no cliente como no servidor, evitando que um programa termine abruptamente (*crash*) quando a outra parte é desligada.
- Usar a função *setsockopt(..., SO_REUSEADDR, ...)* para fazer com que o servidor consiga fazer *bind* a um porto usado anteriormente e registado pelo *kernel* como ainda ocupado. Isto permite que o servidor seja reinicializado rapidamente, sem ter de esperar o tempo de limpeza da tabela de portos usados, mantida pelo *kernel*.
- Caso algum dos pedidos não possa ser atendido devido a um erro, o servidor vai retornar {*OP_ERROR*, *CT_NONE*} independentemente do tipo de erro.

6 Ficheiros disponibilizados

Para a realização deste projeto serão disponibilizadas versões funcionais dos módulos que foram solicitados no projeto 1: *data.o* e *list.o*. Os grupos que não tenham conseguido concluir com sucesso o desenvolvimento do projeto 1 podem utilizar esta módulos. Também são fornecidos os ficheiros *.h* apresentados neste enunciado, bem como o ficheiro *sdmessage.proto*.

Finalmente, como já referido, são ainda disponibilizados os binários de um cliente e de um servidor, que podem ser usados para testes. O protocolo de comunicação implementado nestes dois binários é o que está descrito neste enunciado. Assim, se os programas desenvolvidos pelos grupos respeitarem o protocolo, a interoperabilidade estará assegurada.

7 Makefile

Deve ser construído um *Makefile* que permita compilar os dois programas e que tenha, pelo menos, os seguintes *targets*:

- **all**: criar (pelo menos) os targets *liblist*, *list_client* e *list_server*;

- **liblist:** Compilar os ficheiros `data.c` e `list.c`, criando os respetivos ficheiro objeto, e colocando-os numa biblioteca estática **liblist.a** (usar o comando **ar** com as opções `-r, -c e -s: ar -rcs liblist.a data.o list.o`. Caso sejam usados os ficheiros objeto do projeto 1 disponibilizados aos alunos, deverão ser estes a ser colocados na biblioteca;
- **list_client:** Compilar todo o código que pertence ao cliente, criando a aplicação **list_client** (ligar todos os ficheiros objetos necessários e a biblioteca `liblist.a`);
- **list_server:** Compilar todo o código que pertence ao servidor, criando a aplicação **list_server** ligar todos os ficheiros objetos necessários e a biblioteca `liblist.a`);
- **clean:** Remover todos os ficheiros criados pelos *targets* acima. Nota: se forem usados os ficheiros objeto do projeto 1 disponibilizados aos alunos, estes não precisam de ser removidos pelo `clean`.

8 Entrega

A entrega do projeto 2 tem de ser feita de acordo com as seguintes regras:

1. Colocar todos os ficheiros do projeto, bem como o ficheiro README mencionado abaixo, num ficheiro com compressão no formato ZIP. O nome do ficheiro será **grupoXX-projeto2.zip** (XX é o número do grupo).
2. Submeter o ficheiro **grupoXX-projeto2.zip** na página da disciplina no moodle da FCUL, utilizando a atividade disponibilizada para tal. Apenas um dos elementos do grupo deve submeter e todos os elementos têm de confirmar a submissão.

O ficheiro ZIP deverá conter uma diretoria cujo nome é **grupoXX**, onde **XX** é o número do grupo. Nesta diretoria serão colocados:

- o ficheiro README, onde os alunos podem incluir informações que julguem necessárias (e.g., limitações na implementação);
- diretorias adicionais, nomeadamente:
 - `include`: para armazenar os ficheiros `.h`;
 - `source`: para armazenar os ficheiros `.c`;
 - `object`: para armazenar os ficheiros objeto;
 - `lib`: para armazenar bibliotecas;
 - `binary`: para armazenar os ficheiros executáveis.
- um ficheiro `Makefile` que satisfaça os requisitos descritos na Secção 7. Não devem ser incluídos no ficheiro ZIP os ficheiros objeto (`.o`) ou executáveis que são construídos pelo `Makefile`. Caso sejam usados os ficheiros objeto do projeto 1 disponibilizados aos grupos, estes **devem** ser incluídos no ficheiro ZIP.

Na entrega do trabalho, é ainda necessário ter em conta que:

- **Se não for incluído um `Makefile`, se o mesmo não satisfizer os requisitos indicados, ou se houver erros de compilação (isto é, se não forem criados os ficheiros objeto e executáveis), o trabalho é considerado nulo.** Na página da disciplina, no Moodle, podem encontrar vídeos e documentos do utilitário `make` e dos ficheiros `Makefile`. Aconselha-se, em particular, que seja usado como base o `Makefile` de exemplo disponibilizado na primeira aula TP.
- Todos os ficheiros entregues devem começar com um cabeçalho com três ou quatro linhas de comentários a dizer o número do grupo e o nome e número dos seus elementos.
- Os programas são testados no ambiente dos laboratórios de aulas, pelo que se recomenda que os alunos testem os seus programas nesse ambiente.

O prazo de entrega é dia 26/10/2025 até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

9 Autoavaliação de contribuições

Cada aluno tem de preencher no Moodle um formulário de autoavaliação das contribuições individuais de cada elemento do grupo para o projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário **sofrem uma penalização na nota de 10%.**
- Caso existam assimetrias significativas entre as respostas de cada elemento do grupo, o grupo poderá ser chamado para as explicar.

- Se as contribuições individuais forem diferentes, isso será refletido na nota de cada elemento do grupo, levando à atribuição de notas individuais diferentes.

O prazo de preenchimento desde formulário é o mesmo que a entrega do projeto (26/10/2023, até às 23:59 horas).

10 Bibliografia

[1] B. W. Kernighan, D. M. Ritchie, C Programming Language, 2nd Ed, Prentice-Hall, 1988.

[2] Wikipedia. Linked List. https://en.wikipedia.org/wiki/Linked_list.

[3] <https://developers.google.com/protocol-buffers/docs/overview>